SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Performance prediction of iterator based database systems with decision trees

Hoang-Nam Tran

SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Performance prediction of iterator based database systems with decision trees

Performancevorhersage iteratorbasierter Datenbanksysteme mit Entscheidungsbäumen

Author:	Hoang-Nam Tran
Examiner:	Prof. Dr. Thomas Neumann
Supervisor:	Maximilian Rieger, M.Sc.
Submission Date:	27.04.2025

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 27.04.2025 Hoang-Nam Tran

Acknowledgments

Abstract

Contents

Acknowledgments **iv**

Abstract **v**

1 Introduction **1**
1.1 Section . . . 1
1.1.1 Subsection . . . 1

2 Theoretical Background **3**
2.1 Databases and Query Execution . . . 3
2.1.1 Iterator-Based Execution (PostgreSQL) . . . 4
2.1.2 Compiled Query Execution (Umbra) . . . 5
2.1.3 Pipelines and Pipeline Breakers . . . 6
2.2 Benchmarking and Evaluation Metrics . . . 7
2.2.1 Benchmarking . . . 7
2.2.2 Error Metrics . . . 8
2.3 Performance Prediction Models . . . 8
2.3.1 Traditional Cost Models vs. Learned Approaches . . . 8
2.3.2 Neural Networks . . . 9
2.3.3 Decision Trees . . . 10
2.3.4 T3 . . . 11
2.4 Related Work . . . 14

3 Methodology **17**
3.1 Problem Statement and Prediction Target . . . 17
3.2 Data and Preprocessing . . . 17
3.3 Plan Representation . . . 18
3.4 Pipeline Decomposition . . . 18
3.5 Feature Vector Design . . . 19
3.6 Training . . . 20

4 Implementation Details **22**
4.1 Plan Conversion and Pipeline Decomposition . . . 22

4.2 Feature Vector Construction 22

5 Evaluation **26**
5.1 Generalization and Frequency Distribution 26
5.2 Comparison to Other Models 27
5.3 Ablation Study 31

6 Future Work **36**
6.1 Dedicated Benchmarking and Training Data 36
6.2 Feature Vector Extensions 37
6.3 Prediction Latency 37

7 Conclusion **39**
7.1 Section 39
7.1.1 Subsection 39

Abbreviations **41**

List of Figures **42**

List of Tables **43**

1 Introduction

ABSTRACT FEHLT

RESEARCH QUESTION. UND DANN DIESE AUFGREIFEN IN EVALUATION AM ANFANG.

1.1 Section

Citation test [**latex**].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ Technical University of Munich (TUM), TUM

For more details, see the documentation of the `acronym` package[1].

1.1.1 Subsection

See Table 7.1, Figure 7.1, Figure 7.2, Figure 7.3.

Table 1.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

[1] `https://ctan.org/pkg/acronym`

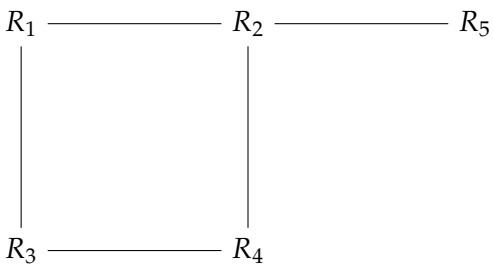

Figure 1.1: An example for a simple drawing.

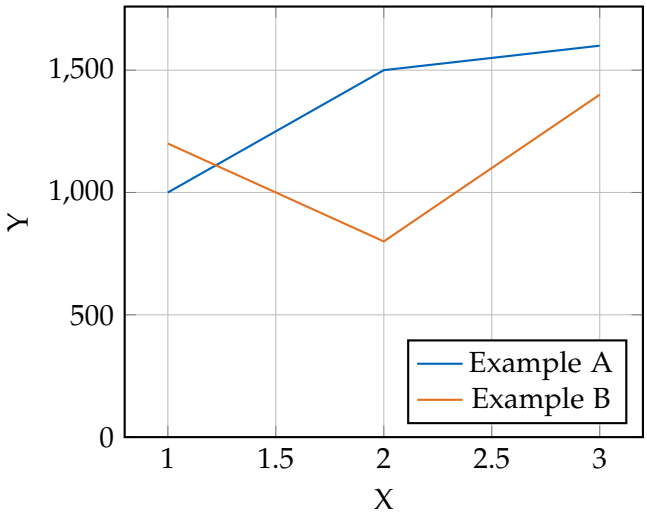

Figure 1.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 1.3: An example for a source code listing.

2 Theoretical Background

2.1 Databases and Query Execution

A *database management system* (DBMS) is software that stores, organizes, and retrieves structured data. Users interact with such systems primarily through *Structured Query Language* (SQL), a declarative language that lets users specify what data they want rather than how to retrieve it [**silberschatz2019**]. For example, a simple SQL query might select all orders placed by a particular customer by joining an `orders` table with a `customers` table and filtering by a customer name. Because SQL describes the desired result and not the retrieval procedure, the DBMS is free to choose any execution strategy that produces the correct answer.

When a DBMS receives an SQL query, it processes the query in three successive phases [**silberschatz2019**]:

1. **Parsing and translation.** The SQL text is checked for syntactic correctness and translated into an internal relational-algebra expression. During this step, the parser also expands any *view* references — a view is a named, stored query that acts as a virtual table — by substituting them with their underlying relational-algebra definition.

2. **Optimization.** The *optimizer* takes the relational-algebra expression and uses statistical summaries of the stored data to evaluate many possible *execution plans*. A plan is a tree of physical *operators* — such as table scans, joins, sorts, and aggregations — along with instructions for how each operator should be executed (e.g., whether to use hash tables or nested loops).

   One key input to the optimizer is the *cardinality* of each operator: the number of rows (tuples) it is expected to produce or consume. Since the query has not yet been executed, the optimizer relies on *estimated cardinalities*, derived from table sizes and column value distributions. These estimates directly influence which plan the optimizer considers cheapest and therefore selects.

   The *actual cardinality* — the true number of rows processed at runtime — only becomes known after execution. Estimated and actual cardinalities can differ substantially, which is a recurring challenge in performance prediction [**leis2018job**].

3. **Evaluation.** The query-execution engine runs the chosen execution plan against the stored data and produces the query result. Most DBMSs provide dedicated commands to extract and inspect the chosen execution plan.

The query plan is central to performance, since the optimizer's choice of operators and their ordering directly determines how much work the DBMS has to perform. Different DBMSs implement these phases differently, and especially the execution step varies widely. *PostgreSQL*, for instance, executes plans using an iterator-based model described below. *Umbra*, by contrast, compiles query plans into native machine code at runtime [**umbra2020**]. These architectural choices have a large impact on query execution time [**neumann2011**].

2.1.1 Iterator-Based Execution (PostgreSQL)

PostgreSQL (PG) uses the *iterator model* for query execution, which is also known as the *Volcano model* or *pull-based execution* after the system that popularized it [**graefe1994**].

In this model, every operator in the query plan tree exposes a uniform three-call interface: `open()`, `next()`, and `close()`. The central call is `next()`: when invoked, the operator produces and returns the next output tuple (i.e., one row of data). If no more tuples are available, the operator signals end-of-stream. The `open()` call initializes the operator and its state, and `close()` releases any resources held.

Operators are composed into a tree that mirrors the query plan. The root operator drives execution: each time it needs another tuple, it calls `next()` on its child operator. That child, in turn, calls `next()` on its own children, and so on, all the way down to the leaf operators — typically table scans — which actually read data from disk or memory. Tuples thus *flow upward* through the tree one at a time, on demand. This is the defining property of pull-based execution: no operator pushes data forward; instead, every operator waits to be asked.

As a concrete example, consider a query that scans a table, filters rows by a predicate, and then sorts the result. The tree would consist of a sort operator at the root, a filter operator as its child, and a sequential scan at the leaf. When execution starts, the sort operator repeatedly calls `next()` on the filter, which in turn calls `next()` on the scan, until all matching rows have been collected. Only then does the sort operator produce its first output tuple.

PG exposes this plan tree through the `EXPLAIN` and `EXPLAIN ANALYZE` commands. They are prepended to any SQL statement and cause PG to return the query plan instead of, or in addition to, the query result. `EXPLAIN` alone shows the chosen plan with the optimizer's cost and cardinality estimates for each node, without executing the query. `EXPLAIN ANALYZE` additionally executes the query and annotates every node with the actual number of rows produced and the actual time

spent [**postgresql_docs_query_path**]. This makes `EXPLAIN ANALYZE` output the primary source of per-operator timing information in PG and is also the format used by the ZeroShot benchmark that this thesis builds on [**zeroshot**].

A key advantage of the iterator model is its simplicity and composability. New operators can be added independently, as long as they implement the three-call interface [**graefe1994**]. However, this design also has a well-known downside: calling `next()` individually for every single tuple introduces per-tuple function call overhead and hurts CPU branch prediction and cache behavior [**neumann2011**]. This is one reason why more recent systems like Umbra take a different approach, as described in the next subsection.

2.1.2 Compiled Query Execution (Umbra)

Umbra is a relational DBMS developed at the Technical University of Munich that is designed for high performance on modern hardware [**umbra2020**]. Unlike PG's interpreter-based approach, Umbra takes a fundamentally different execution strategy: it compiles each query plan into native machine code at query time [**umbra2020**].

The motivation for this is to eliminate the per-tuple overhead of the iterator model. Recall that in PG's pull-based execution, every single tuple causes a chain of `next()` calls to travel through the operator tree. This introduces substantial function call overhead and makes it difficult for the CPU to predict branches and keep data in cache. In a compiled system, these operator boundaries dissolve. Instead of calling functions for each tuple, Umbra generates a single piece of native machine code that fuses the logic of multiple operators into tight loops. Data is *pushed* forward through the code rather than pulled upward through function calls [**neumann2011**].

Concretely, the compilation model works as follows [**neumann2011**]. Each operator participates in a `produce/consume` interface, which is a code-generation protocol invoked once at query compile time. Code generation starts at the root, which calls `produce()` on its child; this propagates downward until the leaf scan is reached. The scan emits a `for`-loop over its tuples and, for each tuple, calls `consume()` on its parent. Each `consume()` invocation emits that operator's processing logic inline — a predicate check, a buffer write, etc. — and then calls `consume()` further up. The result is a compact code fragment: a single `for`-loop from the scan, with the logic of every subsequent operator fused inside it, and no function-call boundaries between operators.

In the scan-filter-sort example from the previous subsection, the first fragment is exactly such a loop. It scans tuples, checks the filter predicate inline, and writes qualifying tuples into a sort buffer. The sort, however, cannot produce output until all input is accumulated. Therefore, it acts as a materialization boundary, after which a second fragment handles the sorted output. These materialization boundaries between

fragments are the concept of *pipeline breakers*, introduced in the next subsection.

At runtime, only the generated code runs. The scan drives execution by iterating over tuples, and each tuple is processed through the entire pipeline inline. This is the push-based contrast with PostgreSQL: in PG, the top-most operator drives execution by issuing `next()` calls all the way down the tree for every single tuple at runtime.

Furthermore, instead of always compiling, Umbra employs an *adaptive compilation strategy* [**umbra2020**]. Each pipeline step is first interpreted via an efficient bytecode virtual machine, avoiding compilation overhead entirely for cheap or short-running steps. For steps where compilation is beneficial, Umbra translates its custom intermediate representation (IR) into LLVM IR [**umbra2020**]. LLVM (Low Level Virtual Machine) is a general-purpose compiler framework that then produces highly optimized native machine code [**neumann2011**].

As a concrete consequence for this thesis: *T3*, the performance prediction model this work builds on, was originally designed for Umbra and therefore relies on Umbra's query plans and their structure [**t3**]. The key structural concept that Umbra's compilation naturally exposes is the notion of *pipelines*, which is described next.

2.1.3 Pipelines and Pipeline Breakers

The code fragments described above correspond to what are formally called *pipelines* — a central concept in compiled query execution and the structural basis of T3's prediction approach [**t3**].

A *pipeline* is a maximal sequence of operators that can process tuples in a fully streaming fashion, without any need to fully materialize an intermediate result. Within a pipeline, each tuple produced by the scan at the bottom flows immediately through all subsequent operators — being filtered, transformed, or joined — without waiting for any other tuple.

Moreover, a *pipeline breaker* is an operator that interrupts this streaming flow, because it must see *all* of its input before it can produce any output. The following are typical examples of pipeline breakers [**t3**]:

1. **Sort.** A sort operator cannot emit a single sorted tuple until it has collected and sorted all input tuples.

2. **Aggregation.** A group-by operator must accumulate all input tuples into groups before it can emit any aggregated result.

3. **Hash join (build side).** The build side of a hash join must be fully read into a hash table before any tuple from the probe side can be matched. The build side therefore forms its own pipeline, terminating at the hash table.

Not every operator is a pipeline breaker. A filter or a projection passes each tuple through without any materialization. The probe side of a hash join — where tuples are looked up in an already-built hash table — behaves the same way. Therefore, a single pipeline can contain multiple consecutive join probe stages.

Thus, a query plan decomposes into a directed acyclic graph (DAG) of pipelines. Each pipeline starts either at a base table scan or by reading the materialized output of a prior pipeline breaker, processes tuples in a streaming fashion, and ends either at the next pipeline breaker or at the final output. The left side of Figure 2.1 illustrates such a pipeline decomposition.

2.2 Benchmarking and Evaluation Metrics

2.2.1 Benchmarking

Section 2.1 described how a DBMS processes SQL queries and produces query plans. To evaluate how well a performance prediction model estimates the execution time of such plans, its predictions must be compared against real, measured runtimes. Benchmarking provides a standardized framework for this.

A benchmark consists of two components: a *database instance* — a schema with populated data — and a *workload* of representative SQL queries [**silberschatz2019**]. Each query in the workload is executed on the target DBMS and its wall-clock runtime is recorded. Since execution time depends directly on the underlying hardware — CPU speed, available memory, and storage throughput — benchmark results are always specific to the machine they were obtained on. Absolute runtimes from different machines are therefore not directly comparable. The goal of a benchmark is instead to provide a fixed, reproducible environment in which different models or systems can be evaluated under identical conditions [**silberschatz2019**].

Join Order Benchmark (JOB). An example of a benchmark is the *Join Order Benchmark (JOB)*, also called JOB-full. We will use it as an evaluation benchmark in this thesis. JOB uses the real-world *IMDB (Internet Movie Database)* dataset and contains 113 complex queries, some involving up to 17 joins. Unlike synthetic benchmarks (those using artificially generated data), the IMDB dataset contains naturally occurring correlations and skewed value distributions. Therefore, it is a challenging and realistic workload and JOB is widely used in query performance prediction research [**leis2018job**].

2.2.2 Error Metrics

Evaluating a performance prediction model requires a metric that captures how far predicted runtimes deviate from actual ones. Two aspects matter: the *accuracy* on individual predictions and the *distribution* of errors across the full workload.

Q-Error. A common metric for cardinality estimation is the *Q-Error*, defined as [**qerror**]:

$$\text{Q-Error}(\hat{t}, t) = \max\left(\frac{\hat{t}}{t}, \frac{t}{\hat{t}}\right),$$

where $\hat{t}$ is the predicted value and t is the true value. A Q-Error of 1 is a perfect prediction. Furthermore, if a query takes 1 s but the model predicts 3 s, the Q-Error is 3; the same Q-Error results if the model predicts 0.33 s instead. Hence, over- and underestimates of the same factor are penalized equally, regardless of the absolute runtime magnitude. T3 uses Q-Error to evaluate runtime predictions [**t3**], and we follow the same approach in this thesis.

Summarizing the Error Distribution. A single Q-Error value describes the accuracy on one query. To characterize a model's behavior across a whole workload, we use three summary statistics: First, the *median* (p50) gives the typical error: half of all queries are predicted within this factor. Second, the *90th percentile* (p90) describes tail behavior: 90% of queries fall at or below this Q-Error. Finally, the *average* Q-Error reflects the mean multiplicative error across all queries, but is more sensitive to a small number of large outliers than the percentile values.

Consequently, these three statistics provide a comprehensive picture of a model's prediction quality across the benchmark workload.

2.3 Performance Prediction Models

2.3.1 Traditional Cost Models vs. Learned Approaches

A *cost model* is what the query optimizer uses to compare candidate execution plans and select the one it expects to be cheapest [**silberschatz2019**]. It assigns a cost estimate to each plan based on operator types, estimated cardinalities, and table statistics such as column histograms. Subsequently, models that predict actual query execution time are also referred to as *performance prediction models* or *learned cost models* [**t3**; **zeroshot**].

Traditional cost models consist of hand-crafted mathematical formulas that are built into the DBMS optimizer. PostgreSQL's built-in estimator is a representative example: it computes cost scores from table sizes, index properties, and planner

statistics without ever observing real execution times [**postgresql_docs_planner_stats**]. These formulas rely heavily on cardinality estimates — and as described in Section 2.1, those estimates can be orders of magnitude off. Since cost estimates are derived from cardinality estimates, such errors propagate directly into the cost model and can cause the optimizer to select suboptimal plans [**leis2018job**].

This limitation motivated *learned approaches*: rather than hand-crafted formulas, a model is trained on historical query executions to learn the mapping from plan features to runtime directly from data [**zeroshot**]. Learned models can capture complex, hardware-specific execution patterns that are difficult to express analytically. The following two subsections introduce the two main families of learned models relevant to this thesis: neural networks and decision trees.

2.3.2 Neural Networks

Neural networks are a class of machine learning models that learn to approximate functions from data by composing multiple layers of parameterized transformations [**deeplearning**]. At each layer, the network takes a *vector* — a fixed-length array of numbers — as input, multiplies it by a learned weight matrix, adds a bias, and applies a nonlinear activation function to produce a new vector. This new vector is then passed to the next layer as its input. By stacking many such layers, the network learns increasingly abstract *representations* of the input: intermediate vectors that capture relevant patterns for the prediction task [**deeplearning**]. The final layer produces the network's output — in the case of performance prediction, a single predicted runtime value. They are valued for their flexibility: given sufficient training data, they can capture highly nonlinear relationships between inputs and outputs. Hence, they are attractive for query performance prediction, where the relationship between a plan's structure and its execution time is complex and hardware-specific [**t3**].

Neural Network Architectures for Query Plans. A query plan is a tree, and encoding a variable-depth tree into a fixed-size vector directly loses structural information. Three families of neural architectures have been used to address this.

Tree-structured networks process a query plan bottom-up: each operator's representation is computed from its own features and those of its children, recursively from the leaves to the root. The key idea is *convolution* — the same learned transformation is applied at every operator node regardless of its position, so the model generalizes to plan trees of different shapes and sizes [**deeplearning**; **e2e**; **qppnet**].

Attention-based models allow each operator's representation vector to be updated based on the representation vectors of all other operators in the plan simultaneously, capturing dependencies between operators that are far apart in the tree [**queryformer**].

Graph neural networks represent the full query context — operators, tables, columns, and predicates — as a graph and propagate information through it [**zeroshot**].

Accuracy vs. Prediction Latency. Neural network models can achieve high prediction accuracy. However, they come at the cost of high inference latency. Since a query must wait for its prediction before being scheduled, this latency adds directly to every query's execution time — a particular problem for short-running queries [**t3**].

ZeroShot. ZeroShot [**zeroshot**] is a neural cost model that addresses a key limitation of workload-driven approaches: such models must be retrained from scratch for every new database, requiring costly collection of training queries. ZeroShot instead trains a single model across a variety of databases and then transfers it to unseen ones without any retraining [**zeroshot**].

The key enabler is a *transferable query representation*: the query plan is encoded as a graph where all node features are chosen to be meaningful across databases — using predicate structure and data types rather than database-specific literals or column names [**zeroshot**]. Representations are propagated bottom-up through the plan graph, and the resulting encoding of the root node is passed to a final network that predicts the runtime [**zeroshot**].

This thesis uses the training data published alongside ZeroShot as its training data source.

2.3.3 Decision Trees

A *decision tree* is a model structured as a tree where each internal node tests a condition on one feature of the input vector and routes the input to a child branch based on the outcome [**cart**]. Traversal starts at the root and continues until a leaf is reached, which holds the prediction value. For example, a node might check whether the estimated output cardinality of a pipeline exceeds a threshold and branch accordingly. Since inference only requires evaluating simple conditions and following branches, it is extremely fast.

A single decision tree has limited expressiveness: the number of distinct predictions it can produce is bounded by the number of leaves [**cart**]. *Gradient boosting* addresses this by building an ensemble of trees sequentially [**gradientboosting**]. Each new tree is trained to predict the residual error left by all previous trees. Hence, the ensemble progressively reduces the prediction error, and the final prediction is the sum of all trees' outputs [**gradientboosting**].

LightGBM is a highly efficient gradient boosting framework that implements this ensemble method with optimizations for speed and memory usage [**lightgbm**]. It is

the framework used by T3 [**t3**].

Advantages and Disadvantages. The key advantage of gradient boosted decision trees is *inference speed*: traversing a tree requires only simple comparisons, with no matrix multiplications. This makes them orders of magnitude faster than neural networks at inference time — T3's compiled model achieves around 4 µs per prediction [**t3**].

A well-known disadvantage is *overfitting*. A single deep decision tree will memorize the training data rather than learning generalizable patterns [**cart**]. Gradient boosting partially mitigates this through two mechanisms [**gradientboosting**]. First, the total number of trees is capped: adding more trees eventually starts fitting noise in the training data rather than learning generalizable patterns, so training is stopped early. Second, each tree's contribution to the final prediction is scaled by a small factor (called *shrinkage* or the *learning rate*) — for example, 0.1 — so no single tree can push the prediction too far. Together, these controls reduce the risk that the ensemble memorizes the training data [**gradientboosting**].

Moreover, another limitation is that decision trees require a *flat feature vector* as input and cannot natively handle tree-structured data like query plans [**t3**]. Additionally, because possible prediction values are bounded by the number of leaves, a single decision tree struggles to predict accurately over a very wide range of target values [**t3**]. Both of these limitations must be addressed through careful design, which T3 applies as described in the next subsection.

2.3.4 T3

T3 (Tuple Time Tree) is a compiled decision tree model for query performance prediction that directly addresses both limitations described above [**t3**]. It is designed specifically for Umbra [**t3**] and serves as the base of this thesis. T3 achieves state-of-the-art accuracy while operating at around 4 µs inference latency – roughly four orders of magnitude faster than neural network models such as ZeroShot [**t3**]. This combination is achieved through three main ideas: a *pipeline-based query plan representation*, *tuple-centric prediction targets*, and *model compilation to native machine code* [**t3**].

Figure 2.1 shows a concrete example: a query plan is decomposed into six pipelines (P0–P5), each converted to its own feature vector. The model then predicts a runtime for each pipeline independently. Summing the six predictions yields the total predicted query time of 38 ms.

Pipeline-Based Representation. T3 exploits the pipeline structure of Umbra's compiled execution model introduced in Section 2.1. Because Umbra already partitions

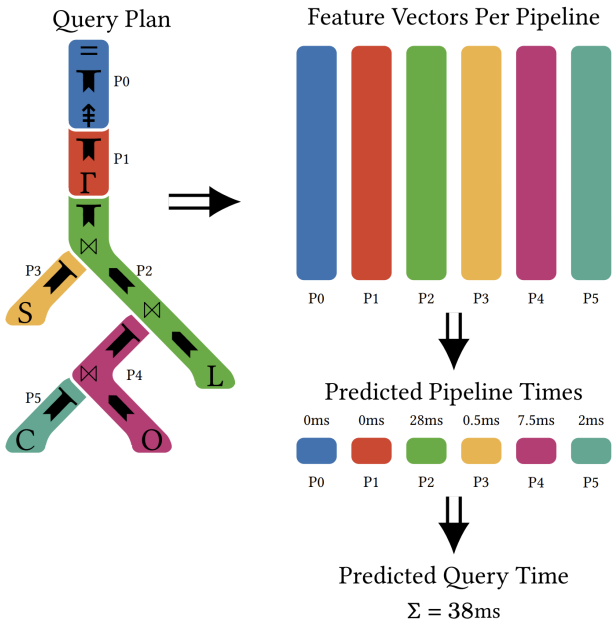

Figure 2.1: T3 prediction process for the TPC-H benchmark, query Q5. The query plan is decomposed into pipelines (P0–P5). Each pipeline is encoded as a flat feature vector. The decision tree predicts an execution time per pipeline. The sum of all pipeline times is the predicted query runtime. Figure from Rieger and Neumann [**t3**].

query execution at pipeline breakers, each pipeline is a self-contained unit with a well-defined input cardinality, a fixed operator sequence, and a clear output.

Therefore, T3 encodes each pipeline as a separate flat feature vector and predicts its execution time individually. The predicted query runtime is the sum of all pipeline times [**t3**]. This decomposition simplifies the prediction problem: instead of capturing the combined behavior of all operators at once, the model focuses on the specific characteristics of each pipeline [**t3**].

Feature Vectors. Different operators do fundamentally different work. For example, a hash join first *builds* a hash table from its left input and then *probes* that table with tuples from its right input. By contrast, a filter simply lets tuples pass through. T3 captures this by assigning each operator a *stage* that describes what it does with tuples [**t3**]:

1. **Build.** Tuples arrive and are materialized (e.g., hash table construction).
2. **Probe.** Tuples are looked up against a materialized structure.
3. **Scan.** An operator reads tuples from storage or a previous result.
4. **Pass-through.** Tuples traverse the operator without materialization (e.g., a filter).

Furthermore, T3 distinguishes which *tuple stream* is involved: the primary input (*in*), the secondary input for binary operators such as joins (*right*), or the output (*out*) [**t3**].

For each stage and stream combination, T3 records up to three numerical features [**t3**]:

1. **Percentage.** The fraction of all tuples at the pipeline start that reach this point.
2. **Size.** The size of a single tuple in bytes, used for materializing operators. T3 derives this from Umbra's *IU* (information unit) list. It is a per-operator record of output attribute names, types, and byte widths. Subsequently, T3 requires database schema information.
3. **Cardinality.** The number of tuples in the stream, used for operators with nonlinear cost such as sorts or hash joins.

The percentage feature is the most important one. An operator that only 1% of tuples reach contributes almost nothing to runtime, regardless of how expensive it would be at full volume [**t3**]. Additionally, T3 records the count of each operator-stage combination that appears in the pipeline. This lets the model handle pipelines where the same operator type occurs more than once [**t3**].

Together, these features give a compact description of each pipeline's load.

Tuple-Centric Prediction Targets. T3's training data spans a runtime distribution from 10^{-15} s to 1 s per tuple – a range of fifteen orders of magnitude [**t3**]. However, a decision tree can output only as many distinct values as it has leaves. Hence, directly predicting pipeline execution times over such a wide range comes with a loss of precision [**t3**].

Instead, T3 addresses this by predicting the expected time to process a *single tuple* through the pipeline. This fits naturally with Umbra's push-based model where tuples flow through the pipeline one by one. Then, the full pipeline time is recovered by multiplying this per-tuple value by the number of tuples scanned at the pipeline's start [**t3**]. This normalization compresses the prediction target range and improves accuracy.

Additionally, the per-tuple times are log-transformed during training ($t' = -\log(t)$), so relative deviations are penalized equally for short and long queries [**t3**].

Training, Compilation, and Generalization. T3 generates its own training data by running 16 distinct query structures on Umbra across multiple database instances and recording the resulting pipeline execution times [**t3**]. It then trains a LightGBM ensemble of 200 gradient boosted trees on these records using the MAPE (Mean Absolute Percentage Error) objective. MAPE minimizes the average relative error across predictions rather than the absolute error. Subsequently, short and long pipelines contribute equally to training [**t3**]. Following the zero-shot paradigm [**zeroshot**], T3 trains on all but one database instance and generalizes to the held-out one without any retraining [**t3**].

After training, the model is compiled to native machine code via the `lleaves` framework, which converts each decision node to exactly one compare instruction and one branch instruction [**t3**]. This compilation step is performed only once after training and reduces inference latency from 22 µs to approximately 4 µs. Hence, T3 is an extremely fast performance prediction model [**t3**].

2.4 Related Work

Main Memory System Cost Models. Performance prediction has been a research challenge for decades. Listgarten and Neimat identify main-memory databases as a disruptive new technology and provide one of the first dedicated cost models for such systems [**listgarten**]. Manegold et al. extend this line of work to hierarchical memory systems, proposing a generic approach to building cost models that account for cache, RAM, and disk access costs [**manegold**].

Both works rely on hand-crafted analytical formulas rather than learned approaches. Thus, they predate the availability of the machine learning techniques used in modern

performance prediction.

Early Learning-Based Methods. Before neural networks became widespread, machine learning was already applied to performance prediction. Gupta et al. use a technique similar to decision trees to predict query time ranges as early as 2008 [**pqr**].

FlatVector [**flatvector**] is an early machine-learning-based approach that encodes each query as a flat feature vector of aggregated statistics and fits a regression model over it. It makes no use of the tree structure of query plans and serves primarily as a simple non-neural baseline.

Workload-Based Neural Network Models. When neural networks became more widely adopted, they were applied to performance prediction for fixed workloads.

MSCN [**mscn**] is a multi-set convolutional network originally proposed for cardinality estimation. It encodes tables, joins, and predicates as separate sets of feature vectors and applies a small neural network to each set before combining their outputs. Also, it can be used as a learned cost model.

QPPNet [**qppnet**] is a plan-structured neural network for query performance prediction. It assigns a separate neural unit to each operator type in the query plan tree, so that each operator's representation is computed from the representations of its child operators. Predictions are aggregated bottom-up to produce a single runtime estimate per query.

E2E [**e2e**] is an end-to-end learned cost estimator based on tree-structured convolutional networks. It encodes the full query plan tree by recursively applying a convolutional layer to parent–child pairs of operators and learns to predict query execution time directly from the encoded plan representation.

NEO [**neo**] is a learned query optimizer that uses a value network to score query plans and guide join ordering. Its internal cost model is a tree-convolutional neural network that encodes query plans similarly to E2E.

QueryFormer [**queryformer**] is a transformer-based cost model that encodes query plans as trees using a tree-structured attention mechanism. By attending over all operator nodes jointly, it can capture long-range dependencies between operators in a plan, which is difficult for purely bottom-up tree-convolutional models.

These workload-specific models achieve strong prediction accuracy but require retraining for every new database [**zeroshot**] and suffer from high inference latency [**stage**].

Modern Zero-Shot Models. A key limitation of workload-based models is that they must be retrained for every new database instance. Zero-shot models address this by training across many databases and transferring to unseen ones without retraining.

Stage [**stage**] is a production performance prediction model built for the entire Amazon Redshift workload. It leverages the accuracy of neural networks while improving latency through a model hierarchy: a query cache is checked first, then a local decision tree, and finally the full neural network as a fallback.

DACE [**dace**] is a database-agnostic cost estimator. It uses an attention-based neural network to encode query plans and is pre-trained across multiple databases. Subsequently, it can generalize to unseen database instances without re-training.

TATA [**tata**] is a transfer learning framework for query plan representations. It transfers a representation trained on one task – such as cost estimation – to other tasks such as query optimization or index selection, reducing dataset collection cost by up to $5\times$.

T3-Based Models with Different Feature Vectors. Wehrstein et al. [**jobcomplex**] also adapted the Umbra T3 model for PostgreSQL. They use a hybrid feature vector that takes the original Umbra feature vector, and maps matching PostgreSQL operators to Umbra operators. On top of that, they add additional features that are specific to PostgreSQL, like the Merge Join operator.

However, their approach does not use fields that account for other relevant PostgreSQL-specific characteristics, like the planned workers or the estimated startup cost fields. Our approach instead constructs a fully PostgreSQL-native feature vector from the ground up. Whether this design leads to better accuracy is investigated in this thesis.

Evaluation of Learned Cost Models. Heinrich et al. systematically evaluate seven state-of-the-art learned cost models (LCMs) – FlatVector, MSCN, E2E, QPPNet, QueryFormer, ZeroShot, and DACE – on three core query optimization tasks. They evaluate join ordering, access path selection, and physical operator selection [**howgoodlcm**].

Their key finding is that while LCMs outperform PostgreSQL's built-in estimator in cost estimation accuracy, this does not automatically translate into better plan selection. They find that high tail errors still cause poor plan choices in downstream optimization tasks. This concerns plan selection, not runtime prediction accuracy, and is therefore consistent with our results. For example, in our evaluation, PostgreSQL's built-in estimator achieves a median Q-error of over 1000 for runtime prediction, far worse than all learned models.

Our thesis focuses on prediction accuracy rather than plan selection, and this work confirms that prediction quality remains a meaningful and open research problem.

3 Methodology

3.1 Problem Statement and Prediction Target

In this thesis, we address the transfer of the T3 runtime prediction approach from Umbra to PostgreSQL parsed plans. The central methodological goal is to preserve T3's good performance [**t3**]. Thus, we use its main concepts and adapt plan conversion and feature extraction to PostgreSQL-specific plan information accordingly.

The final prediction target is total query runtime. Each query plan is first decomposed into pipelines and a feature vector is computed for each pipeline. These per-pipeline feature vectors are then summed into a single query-level feature vector, from which the model directly predicts the total query runtime in one step. The pipeline decomposition is therefore used for feature extraction, not as a separate prediction unit.

3.2 Data and Preprocessing

Regarding the training data, we use the public Zero-Shot benchmark data introduced with the Zero-Shot Cost Model. The benchmark is particularly suitable for this thesis because it consists of real-world datasets (e.g., IMDB) and synthetically generated workloads designed to cover diverse query structures. This includes *SPAJ* (Select-Project-Aggregate-Join) queries with conjunctive predicates. Overall, the benchmark contains 20 databases with large variation in tables, columns, and foreign-key relationships [**zeroshot**].

Having this multitude of databases allows us to test the generalization capabilities of our model. Similar to the T3 paper [**t3**], we can train the model on all but one databases and test on the remaining one. Moreover, as the widely used JOB workload is included, we can compare our performance predictions to other implementations later on. Further, PostgreSQL is the intended database system of our implementation, which avoids the effort of building a separate PostgreSQL benchmark.

Following the benchmark construction described in the ZeroShot paper [**zeroshot**], queries are generated per database and executed on a PostgreSQL DBMS. Then, they are stored as raw traces (i. e., the output of `EXPLAIN ANALYZE`). Next, the authors preprocess the queries to remove very long-running queries (queries above 30 s are

excluded) to keep the benchmark practical and reproducible [**zeroshot**]. A further preprocessing step is to parse the raw traces into structured plan representations. These `parsed_plans` contain the operator nodes found in the raw traces, but organize them in a tree structure with node-wise attributes. For every node, e.g., operator type, estimated/actual cardinalities, widths, and runtimes are stored in the `plan_parameters` field.

The tree structure can be used to create pipelines and the node information will be used to train the model. Consequently, the parsed plans offer a good starting point for our pipeline and we can begin without further preprocessing.

3.3 Plan Representation

We start from one element, i. e., one plan, of the `parsed_plans` array in the benchmark JSON files. An object in the array is a rooted tree: every node carries `plan_parameters` (operator name, estimated and actual cardinalities, estimated tuple width, planner costs, optional filter predicates, timings, and further fields) and a list `children` of child subtrees.

We convert this tree recursively into the same nested operator structure that the original T3 implementation expects for Umbra plans [**t3**]. PostgreSQL operator names are mapped to abstract operators (`tablescan`, `join`, `sort`, and `groupby` nodes).

Regarding joins, the child order is normalized so that it matches T3's pipeline rules: i. e., the PG-build subtree becomes the *left* child and the PG-probe subtree the *right* child.

Since the feature vector uses the PostgreSQL-native features and operators of the `parsed_plans`, we need to keep the original, not converted `plan_parameters` available for the feature extraction. Hence, the converted abstract operator names are only used to match the internal T3 representation for pipeline assignment, but not for the feature extraction.

3.4 Pipeline Decomposition

Umbra plans group operators into *pipelines*: sets of operators that can produce tuples in a pipelined fashion, separated by *pipeline breakers*. Namely these are operators/stages, where data is materialized. For instance, sorting, grouping, or materialization, but also the build side of a hash join [**t3**].

Since PostgreSQL plans do not provide such a pipeline breakdown, we need to derive it from the converted tree. We assign each node to a pipeline by walking the converted tree with the same high-level rules as in the original T3 implementation [**t3**]. Thus, PG's sort and aggregate nodes break pipelines; a PG-materialize node is treated as a

breaker. Joins place the build side in one pipeline and keep the join together with the probe side in the other pipeline.

3.5 Feature Vector Design

We retain the pipeline decomposition for feature extraction rather than opting for a complete query-level feature vector for three reasons. By mirroring T3's pipeline-based feature design [**t3**], we can use the same pipeline-centric features such as operator percentages, which are only well-defined within a pipeline boundary. Further, following T3's ablation study, which includes per-pipeline predictions, we are able to provide a more thorough comparison of prediction granularities in Section 5.3. Finally, PostgreSQL-specific issues affect the per-pipeline and per-tuple approaches in our setting, as discussed in the evaluation (Section 5.3); the decomposition keeps the model extensible so that future work can address those issues.

The original T3 feature mapper for Umbra builds a feature vector from execution phases within each pipeline (e.g. scan, build, probe phases) and from cardinalities and tuple sizes attached to those phases [**t3**]. On the other hand,PostgreSQL plans contain different data and we construct the feature vector for each pipeline after the aforementioned pipeline decomposition.

Once the feature vector is constructed, we can train the model. Our feature vector differs from the original T3 feature vector in the following ways:

1. **PostgreSQL-native operator features.** The `parsed_plans` expose PostgreSQL-native operator nodes. Thus, we add entries for these PostgreSQL-specific operators in the feature vector.

   Regarding stages,T3's stage assignment is derived from Umbra-specific operator types and pipeline positions. There is no direct equivalent mapping for PostgreSQL operators. Hence, we decide against rewriting this logic entirely for PG and do not include stage-based features. Instead, we aggregate features only per operator.

2. **PostgreSQL planner cost estimates.** We include sums and extrema of PostgreSQL's `est_cost` and `est_startup_cost` from `plan_parameters` in the feature vector. These are optimizer-internal estimates of work and startup cost and are an additional PostgreSQL-specific signal that is not available in the Umbra feature vector. Consequently, we can include this information in the feature vector.

3. **Planned parallelism.** Another PostgreSQL-specific signal is the planned parallelism. So, we take the sum of `workers_planned` over all operators in the pipeline. Hereby, we include a parallelism signal in our feature vector.

4. **Sizes.** T3 uses the size of output tuples in bytes for materializing operators [**t3**]. In Umbra, T3 derives the tuple size by summing the per-attribute byte sizes from the operator's IU (information unit) list, giving a precise estimate of the tuple byte width. PostgreSQL provides a direct semantic equivalent: `est_width`, the planner's estimated average output row width in bytes per node. Subsequently, we use `est_width` as the tuple size for each operator. The main difference to the Umbra approach is that `est_width` is a pre-aggregated planner estimate rather than a sum of per-attribute type metadata.

In the Umbra feature vector of T3, rows are indexed by both operator type and execution phase (scan, build, probe, ...) like `HashJoin_Probe...` [**t3**]. Now with the PostgreSQL implementation, we miss out on the execution phase dimension.

Whereas not an equivalent substitute, we replace this information with a pipeline-level dimension and keep the operator-level features for the individual operators. The pipeline-level block produces additional metadata in a compact manner, e.g., by using averages and sums over operators. For instance, we consider planned parallelism, width, and cost estimates per pipeline and not per operator. Subsequently, the feature vector stays more compact.

Each pipeline's feature vector consists of two concatenated blocks: pipeline-level aggregates and an operator-level block.

In total, the feature vector is fixed and has 153 dimensions (23 pipeline-level features and 130 operator-type features). Empty features are zeroed to keep the feature vector dimension.

Regarding the cardinalities, either the actual or the estimated cardinalities are used in the feature vector, depending on which is selected in training.

Once the feature vectors for all pipelines are constructed, we get the feature matrix for the whole plan.

3.6 Training

We keep T3's LightGBM decision tree setup [**t3**] and use the newly constructed PostgreSQL-native feature matrix. A main difference is that we predict the total query runtime directly, rather than predicting individual pipeline durations.

The training label is the total observed query runtime. The model input is formed by summing all per-pipeline feature vectors of a query into a single query-level feature vector. This per-query prediction approach was found to perform best for our PostgreSQL setting, as demonstrated in the ablation study (see Section 5.3).

In T3 [**t3**], the training target is the per-tuple execution time for each individual pipeline instead, and the predicted pipeline times are summed to obtain the query

runtime. We evaluate all three granularities – per-query, per-pipeline, and per-tuple – in the ablation study and discuss why per-query works best for our case.

Our model uses gradient boosted decision trees as implemented in LightGBM [**lightgbm**]. The booster is trained with a mean absolute percentage error (MAPE) objective on the training rows, in line with the original T3 work [**t3**].

Furthermore, we use the same training parameters as in the original T3 work [**t3**]. Hence, the pipeline duration targets t are transformed as $-\log(t)$ to stabilize training. Also following T3, we cap the number of boosting rounds at 200 trees to mitigate overfitting.

Regarding the training data, we mirror T3's train/validation split. We sample 80% of the training data for training and 20% for validation [**t3**]. The default seed used for shuffling the training data is 42.

Concerning the train/test split, we match the training intention of the Zero-Shot benchmark [**zeroshot**] and of T3's cross-instance evaluation [**t3**]. Both aim to measure whether the trained model transfers to an unseen database instance. Thus, we use a leave-one-out split on the `parsed_plan` files: one instance is held out as the test set and all other instances form the training data.

4 Implementation Details

4.1 Plan Conversion and Pipeline Decomposition

Figure 4.1 shows such a structured plan representation for a `Hash Join` and its children, directly taken from the ZeroShot benchmark data [**zeroshot**]. Here, data like cardinalities and filter predicates can be retrieved from the `filter_columns` field. Furthermore, PostgreSQL-specific fields like `workers_planned` (planned parallel workers for the node) are available.

In `zeroshot_to_t3.py`, we convert this tree recursively into the same nested operator structure that the original T3 implementation expects for Umbra plans [**t3**]. The PostgreSQL-specific temporary operators `hash` and `materialize` are mapped to the abstract operator `temp`. Consequently, every converted node additionally stores a copy of the original `plan_parameters` under a dedicated `pg` payload. Training and inference, that use PostgreSQL-native features, read from this payload and from the pipeline layout.

T3 also requires database schema information. However, the Zero-Shot data does not provide this information, so a minimal placeholder schema is used instead.

The pipeline assignment is stored in the `analyzePlanPipelines` list of the converted plan. For every pipeline, we also derive start and stop times from the per-node actual timings in `plan_parameters`. These pipeline durations are not used by the primary model, but are available for the per-pipeline and per-tuple prediction variants evaluated in the ablation study (Section 5.3).

4.2 Feature Vector Construction

As PostgreSQL plans contain different data, we use `pg_features.py` to construct the feature vector for each pipeline after the pipeline decomposition. Subsequently, the `plan_parameters` are kept in the converted tree (`pg` payload entry), as we use them for the feature extraction.

Each pipeline's feature vector consists of two concatenated blocks, as Figure 4.2 shows:

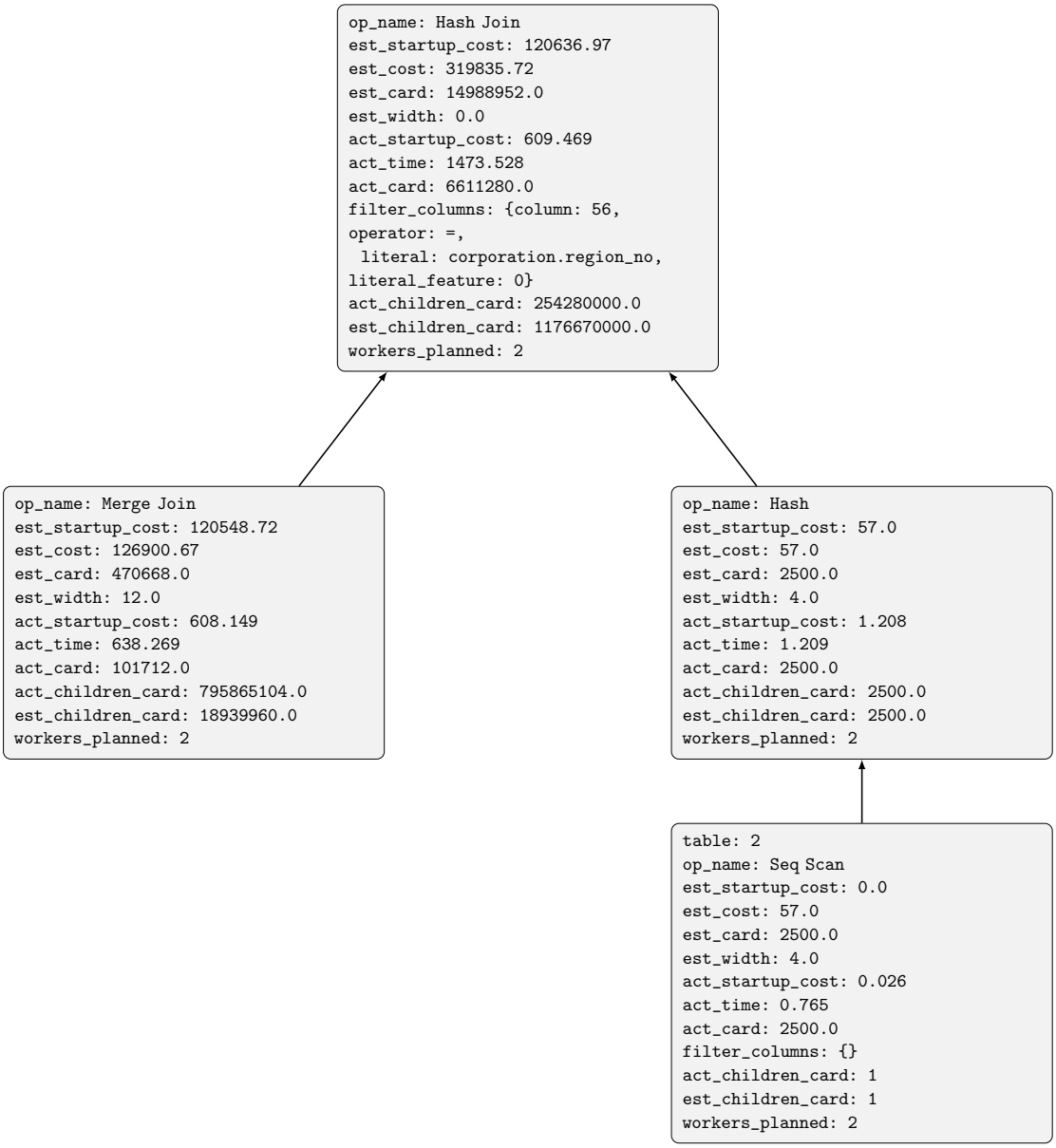

Figure 4.1: Top-node tree cutout of one `parsed_plan` with a `Hash Join` from `credit/workload_100k_s1_c8220`. Child relationships are shown by arrows, and each node box contains selected `plan_parameters` of that node for the displayed cutout.

1. **Pipeline-level aggregates.** From the available `pg` payload, we extract as much distinct information as possible.

   Therefore, we start by summarizing different information categories per pipeline with sum, max, and average statistics. First, the `pg_card_...` fields capture output cardinality summaries.

   Insights about PostgreSQL-specific cost, parallelism, and width features are captured in the `pg_est_cost_...`, `pg_est_startup_...`, `pg_workers_planned_sum` and `pg_est_width_avg` fields.

   Further, the `pg_num_(scan/join/sort/etc.)` family counts operator classes.

   Moreover, the `pg_scan_has_filter` and `pg_filter_(and/or/compare/like/in)` counters enumerate scan predicate attributes.

   Finally, pipeline-context features describe structure using `pg_pipeline_num_ops` for instance.

2. **Operator-level block.** For every operator that occurs in `parsed_plans`, we assign a slice in the feature vector. If multiple operators of the same type occur in the pipeline, we sum the features over all occurences. In Figure 4.2, the excerpt of the operator-type block for the `Nested_Loop join` operator is depicted.

   Depending on the operator type, we include different features. This feature set is derived from T3's Umbra implementation [**t3**]. For example, a join would include different features than a scan, like in the Umbra/T3 approach.

   Regarding the calculations of the features, we follow the Umbra/T3 implementation where adequate. A deviation would be, e. g., how expression percentages are formed due to the different nature of PostgreSQL's filter predicates (CITE).

   Finally, we include the operator count "`const`", which exists for every operator type in both our and the Umbra/T3 implementation [**t3**].

Pipeline-level block (excerpt; first 12 of 23 features)	
`pg_card_sum`	6541.0
`pg_card_max`	6537.0
`pg_est_cost_sum`	16882.95
`pg_est_cost_max`	16877.3
`pg_est_startup_sum`	125.82
`pg_workers_planned_sum`	4.0
`pg_est_width_avg`	10.0
`pg_num_scan`	1.0
`pg_num_join`	1.0
`pg_scan_has_filter`	0.0
`pg_filter_compare_count`	0.0
`pg_pipeline_num_ops`	2.0
Operator-type block (excerpt; Nested_Loop slice; 130 entries in full)	
`Nested_Loop_const`	1.0
`Nested_Loop_in_percentage`	0.11473152822395594
`Nested_Loop_right_card`	98378.0
`Nested_Loop_out_percentage`	0.008260670032124828
(remaining operator-type entries omitted)	

Figure 4.2: Excerpts of the pipeline-level block (first twelve features) and of the `Nested_Loop` operator entries in the operator-type block. They are from one pipeline of query `index_workload_100k_s2_c8220_890` on the TPC-H database. The full vector has 153 dimensions; remaining pipeline and operator entries are omitted.

5 Evaluation

We provide an evaluation of the performance of our model and compare it to other work. Unless otherwise noted due to the nature of the evaluation, all results are evaluated on actual cardinalities. Therefore, we detach our results from the ongoing issue of accurately estimating cardinalities [**leis2018job**].

Environment. Model training and evaluation for the results reported in this thesis were run on a personal laptop: an Apple MacBook Pro (16-inch, 2021) with the Apple M1 Pro processor (10 CPU cores, 16 GPU cores), 16 GB memory, and a 512 GB SSD, under macOS 15.7.3.

5.1 Generalization and Frequency Distribution

Accuracy and Generalization across Databases. Figure 5.1 shows the generalization capabilities of our model across different, unseen database instances. This is ensured by training the model on all databases except the one being tested.

The median Q-Error (p50) stays below 1.33 for all databases, ranging from 1.07 (Walmart) to 1.32 (Basketball), indicating consistently good accuracy on unseen databases. We observe that the p90 and average values show more variation across databases: p90 ranges from 1.30 (Walmart) to 3.32 (Basketball), and the average from 1.13 (Walmart) to 2.92 (Hepatitis). This reflects that a small number of harder-to-predict queries drive up the tail and mean errors for some databases. Despite the increased variation in the average and p90 values, we conclude that the model is able to generalize across unseen database instances. This finding applies to the queries of the ZeroShot training data we used.

Since we trained on the same data as the ZeroShot model, it is reasonable to compare the median values of our model to theirs. As their median values across databases are in the same range [**zeroshot**], we infer that our model has a comparable generalization performance.

Frequency Distribution. Figure 5.2 shows the Q-Error frequency distribution of our model on the JOB benchmark (which we use further in the evaluation). The distribution

is heavily right-skewed: the majority of the 77 queries are predicted with a Q-Error close to 1, with a median of 1.33. However, a small number of queries have considerably higher errors, pulling the average up to 1.89 and the 90th percentile to 3.41, with the worst-predicted query reaching a Q-Error of 7.43. This skewed distribution explains the general pattern visible in Figure 5.1: the p50 stays low and stable across databases, while the average and p90 are more sensitive to a few hard-to-predict outliers.

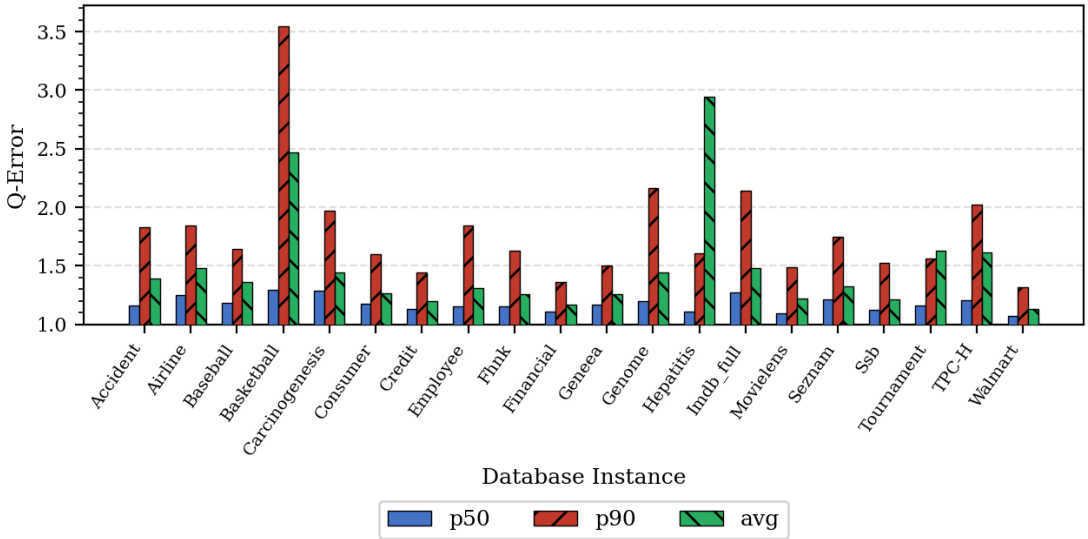

Figure 5.1: Q-Error (p50, p90, and average) of our model evaluated across all database instances. Each database is excluded from the training set when tested and plotted. All other databases are included in the training set.

5.2 Comparison to Other Models

Comparison to T3 (Umbra) and ZeroShot. The original T3 model for Umbra by Rieger and Neumann [**t3**] and the ZeroShot model [**zeroshot**] offer very good generalization performance across unseen databases. Because T3 (Umbra) uses different queries for generalization testing, we cannot directly compare the generalization performance of our model to theirs. But, Rieger & Neumann compared both models on the JOB-Benchmark and reported excellent performance for both models [**t3**].

We also have JOB queries available for evaluation, so we can compare the models on that. In the original comparison, Rieger & Neumann compare the p50, p90, and average Q-Error values. They had all 113 JOB queries available for evaluation.

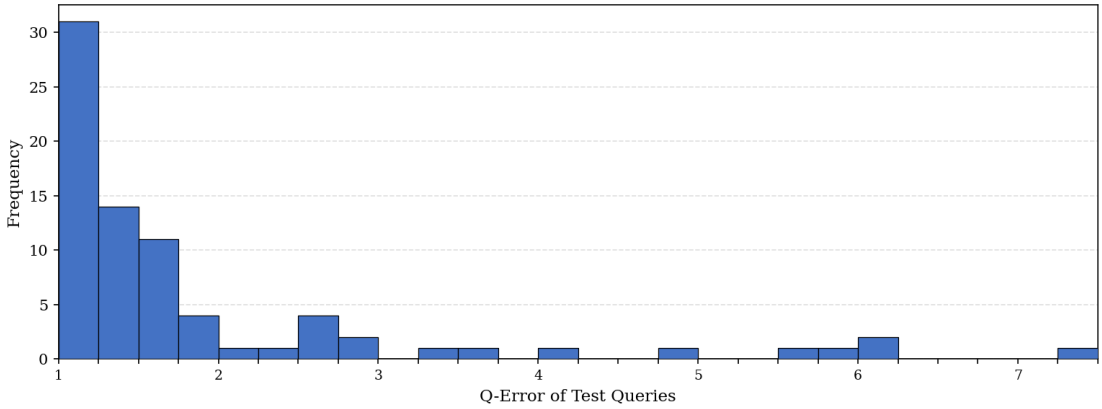

Figure 5.2: Frequency distribution of Q-Error values across all available JOB queries. The model is trained on all databases except IMDB, which is the database of the JOB-Benchmark.

However, the provided ZeroShot training data came with data limitations. Namely, longer-running queries are not included [**zeroshot**], so we only have 77 JOB queries available for evaluation. As the number of total queries differ, it is not sensible to compare the p50 and p90 Q-Error values directly. If the missing queries had a very high Q-Error, the p50 value of our model would be higher than now and using the current p50 value would be misleading. This also applies to the p90 value.

Considering how the median is calculated, we end up with the 57th-best Q-Error value when we take the p75 Q-Error of our model. The 57th-highest Q-Error (p75) also corresponds to the p50 error of the ZeroShot and T3 (Umbra) models. Therefore, even if the missing queries had very high errors, the new p50 value would not get worse than the current p75 value. Hence, we can use it as a lower bound for the median performance of our model and compare it in Figure 5.3.

In this case, our model performs worse than the ZeroShot and T3 (Umbra) models on the JOB-Benchmark. Using the current 57th-best (p75) value for comparison implies a worst-case-scenario for our model. With lower Q-Errors on the missing queries, the new 57th-highest (p50) value would be lower.

To further narrow down the comparison, we also include the average Q-Error values in Figure 5.4. Although we only have 70% of the queries available for the calculation on our model, we still see a notable trend. Decision trees like T3 offer SOTA performance on average for Umbra, followed by ZeroShot. Our model, a T3-based implementation for PostgreSQL, tends to perform worse, but the difference on average is smaller than on percentiles. Thus, it is still is a good starting point.

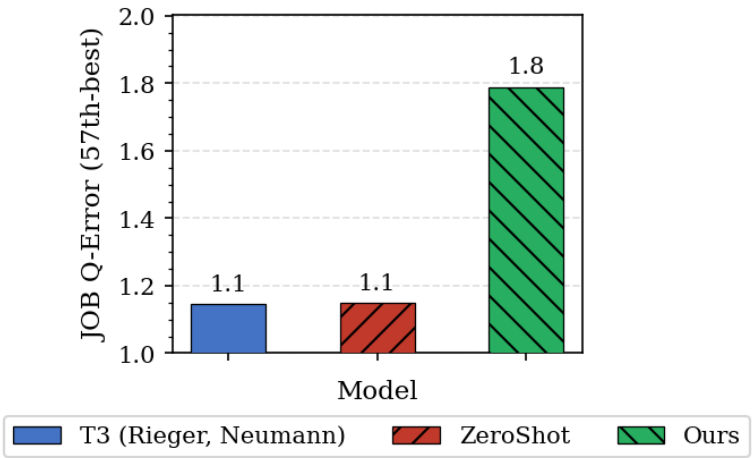

Figure 5.3: 57th-best Q-Error on the Join Order Benchmark for T3 (Umbra), ZeroShot, and our model. Values for T3 (Umbra) and ZeroShot are taken from Rieger & Neumann [**t3**]

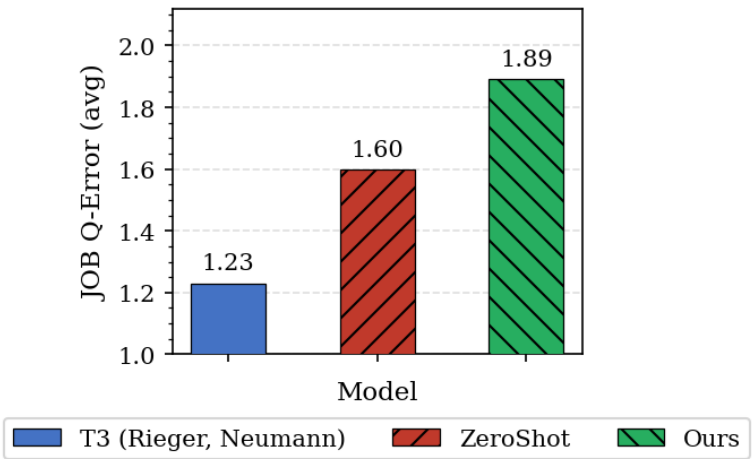

Figure 5.4: Average Q-Error on the Join Order Benchmark for T3 (Umbra), ZeroShot, and our model. Values for T3 (Umbra) and ZeroShot are taken from Rieger & Neumann [**t3**]. Our model's average is calculated over 77 instead of 113 queries.

T3-based Models with Different Feature Vectors. Wehrstein et al. ran the JOB-Benchmark on their T3-based model and reported a median Q-Error of 4.07, using estimated cardinalities [**jobcomplex**]. Like before, they had the full 113 queries at disposal. We again use the 57th-best (p75) Q-Error value for the comparison with our model, this time selecting cardinality estimates to match their setup.

Although we are in the worst-case-scenario, our model clearly outperforms Wehrstein et al.'s model, as Figure 5.5 shows. A key difference between the models is the used feature vector, where only our model uses a completely PostgreSQL-native feature vector. Hence, we demonstrate the relevance of PostgreSQL-native features.

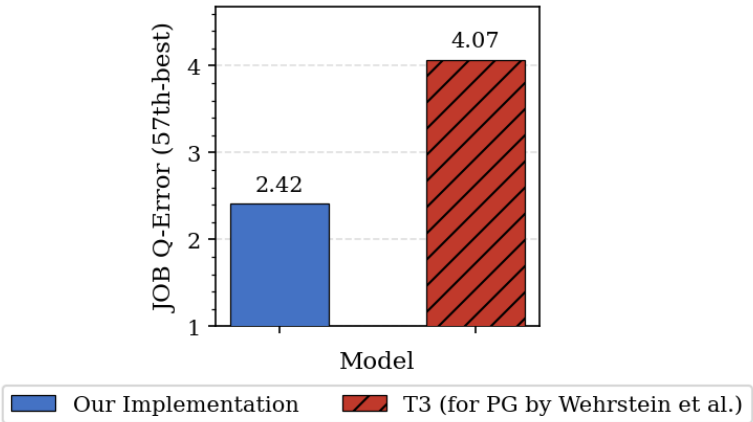

Figure 5.5: 57th-best Q-Error on the Join Order Benchmark. Our model against T3 adapted for PostgreSQL by Wehrstein et al [**jobcomplex**]. Estimated cardinalities.

Further PostgreSQL Prediction Models. Table 5.1 shows the median Q-Error values for different learned cost models on the JOB-Benchmark using PostgreSQL and estimated cardinalities. Once again, we use the 57th-best (p75) Q-Error value for the comparison with our model.

Here, the best performing models are neural networks like ZeroShot [**zeroshot**] and DACE [**dace**], having a median Q-Error below 2. Our model is in the top 4, coming in at 2.42, even when using a worst-case-scenario. The remaining models are neural network-based estimators, such as QPPNet [**qppnet**]. They show a significant accuracy gap to the top performers, with median Q-Errors ranging from 8 to 18. Finally, PostgreSQL's untrained, built-in cost estimator performs worst by a very large margin at a median Q-Error of over 1000, confirming the need for learned approaches. Regarding

learned approaches, not only neural networks can perform, but also decision trees like T3 for PostgreSQL.

Model	Median Q-Error
Postgres (v16)	1015.67
DACE	1.91
E2E	13.21
FlatVector	2.15
MSCN	13.85
NEO	13.20
QPPNet	17.50
QueryFormer	8.30
ZeroShot	**1.58**
Our model	2.42

Table 5.1: Median Q-Error on the Join Order Benchmark (JOB) across different models, data from Wehrstein et al. [**jobcomplex**]. Our model reports the 57th-best (p75) Q-Error value, matching the median of the other models. Estimated cardinalities.

5.3 Ablation Study

Actual vs. Estimated Cardinalities. Figure 5.6 shows prediction accuracy under three cardinality conditions. For each condition, we used the model variant trained under the matching setting. Therefore, for a query from database X, we used the model that was trained on all databases except X and used the corresponding cardinality type. This ensures the correctness of the comparison.

Using exact cardinalities for both training and evaluation gives the best results. This is a setting of our thesis, where we do not investigate the cardinality estimation issue. When, for the models trained on actual cardinalities, we switch to estimated cardinalities for evaluation, accuracy degrades. The median Q-Error increases only moderately, while the p90 and average Q-Error rise more sharply. Hence, it reflects that large cardinality estimation errors lead to large performance prediction errors for a subset of queries, disproportionately affecting the p90 errors and the average [**t3**]. The same asymmetric degradation pattern has been observed for other performance prediction models [**stage**].

Training on estimated instead of exact cardinalities partially compensates for this

effect. The model trained and evaluated on estimated cardinalities achieves a lower average Q-Error than the model trained on exact cardinalities but evaluated on estimated ones. Thus, this demonstrates that the model can somewhat learn to account for the systematic biases of the cardinality estimator [**leis2018job**]. However, the p90 Q-Error stays at a similar, high level in both estimated-cardinality settings, indicating that a small number of heavy outliers remain.

We conclude that incorrect cardinality estimates distort performance prediction accuracy substantially. Cardinality estimation remains an important and unsolved problem [**leis2018job**], and improvements in cardinality estimation would directly benefit performance prediction.

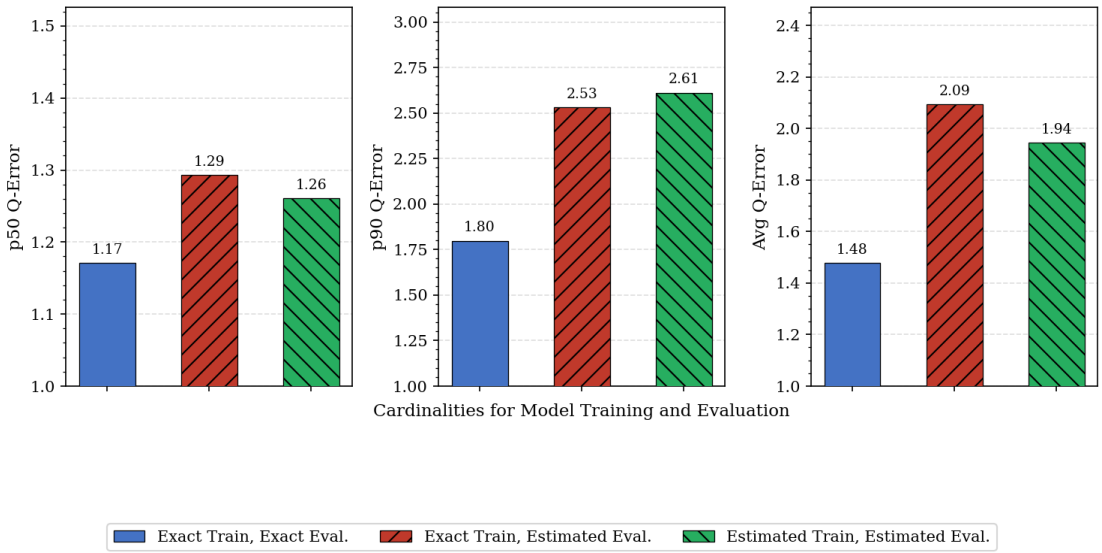

Figure 5.6: Q-Error (p50, p90, and average) over all queries under three cardinality settings: trained and evaluated on exact cardinalities, trained on exact but evaluated on estimated cardinalities, and trained and evaluated on estimated cardinalities.

Query vs. Pipeline vs. Tuple Targets. Figure 5.7 compares Q-Error across three prediction granularities: per-query, per-pipeline, and per-tuple. This directly copies the evaluation of Rieger and Neumann [**t3**], who find that finer granularities consistently improve accuracy — with per-tuple being best and per-query being worst. We observe the opposite: the per-query model achieves the lowest Q-Error across all three metrics, while per-pipeline and per-tuple models perform noticeably worse.

The core issue is the reliability of the per-pipeline execution times that can be extracted from PostgreSQL's output in the ZeroShot dataset [**postgresql_docs_query_path**]. In Umbra, pipeline breakers require full materialization before any downstream pipeline can begin scanning their output [**t3**], making pipelines sequential execution units whose individual durations sum to the total query time.

Unlike Umbra, PostgreSQL query plans do not expose pipeline execution times directly. Instead, they must be derived from the per-node durations recorded by `EXPLAIN ANALYZE` [**postgresql_docs_query_path**].

However, PostgreSQL uses the Volcano iterator model [**graefe1994**], which does not execute pipelines as separate sequential units.

For example, a hash join operator (belonging to pipeline P1) builds its hash table by calling `next()` on its input table scan (belonging to pipeline P0) one tuple at a time. The join and the scan therefore execute interleaved over the same wall-clock period: the join is active from its first `next()` call to the moment the hash table is complete, and the scan is active during every individual tuple fetch within that same period. However, the plan shows the total wall-clock interval for each node, so both the join (P1) and the scan (P0) receive overlapping time intervals — even though they belong to different derived pipelines.

When these node durations are aggregated into per-pipeline times and then summed, the overlapping portion is counted twice, so the sum of pipeline runtimes exceeds the actual query execution time. A model trained to predict query runtime as the sum of pipeline runtimes will therefore over-predict, increasing Q-Error.

As a concrete example, in plan 2 of `tpch/workload_100k_s1_c8220`, pipeline P0 (table scan) is recorded from $t = 946$ms to $t = 2{,}156$ms, giving a span of 1,210ms. Further, Pipeline P1 (table scan, join) is recorded from $t = 0$ms to $t = 2{,}185$ms, giving a span of 2,185ms. Thus, P0 is entirely nested inside P1's time window, so the 1,210ms during which both pipelines are simultaneously open is counted once in P0 and again in P1. When summed, the two spans alone already yield 3,395ms, yet the actual query execution time is only 2,239ms — an overcount of 52%.

The per-tuple approach, which divides the pipeline runtime into per-tuple runtimes, is additionally weakened by a difference in dataset characteristics. In T3, the query execution time span is larger than 10^{-5} s to 10 s [**t3**]. Thus, Rieger and Neumann identify this wide range as the primary motivation for the per-tuple approach. Decision trees have a limited number of discrete leaf values and struggle to predict accurately over such a large range. Hence, normalizing by tuple count compresses the prediction target and improves accuracy [**t3**]. However, in our PostgreSQL dataset, query execution times span a much narrower range, as visible in Figure 5.8.

Subsequently, with a narrower range and per-pipeline targets that are already unreliable, the per-tuple decomposition provides no accuracy benefit and introduces

additional inaccuracies instead.

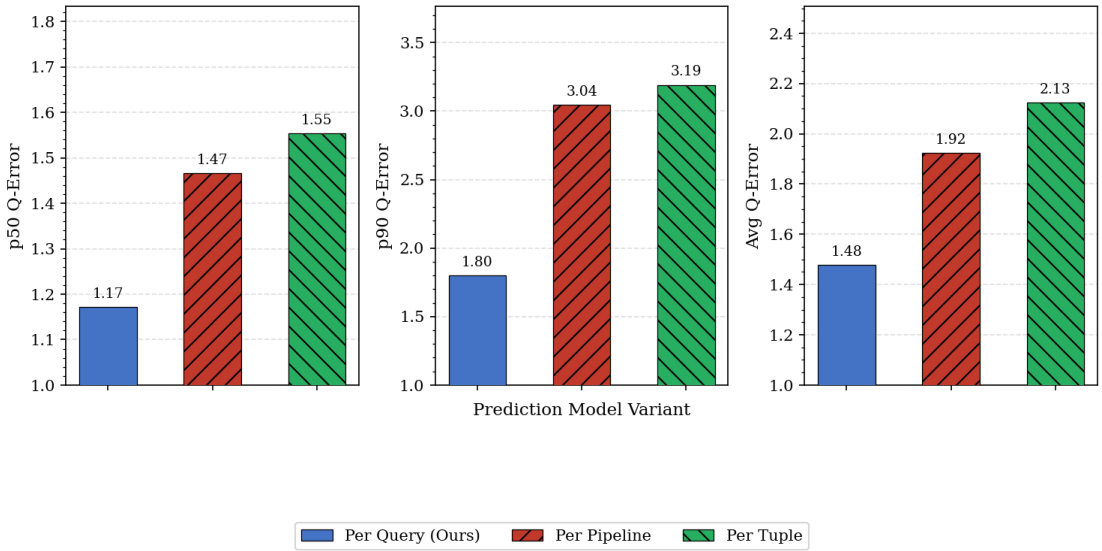

Figure 5.7: Q-Error (p50, p90, and average) for three prediction granularities over all queries: per-query, per-pipeline, and per-tuple.

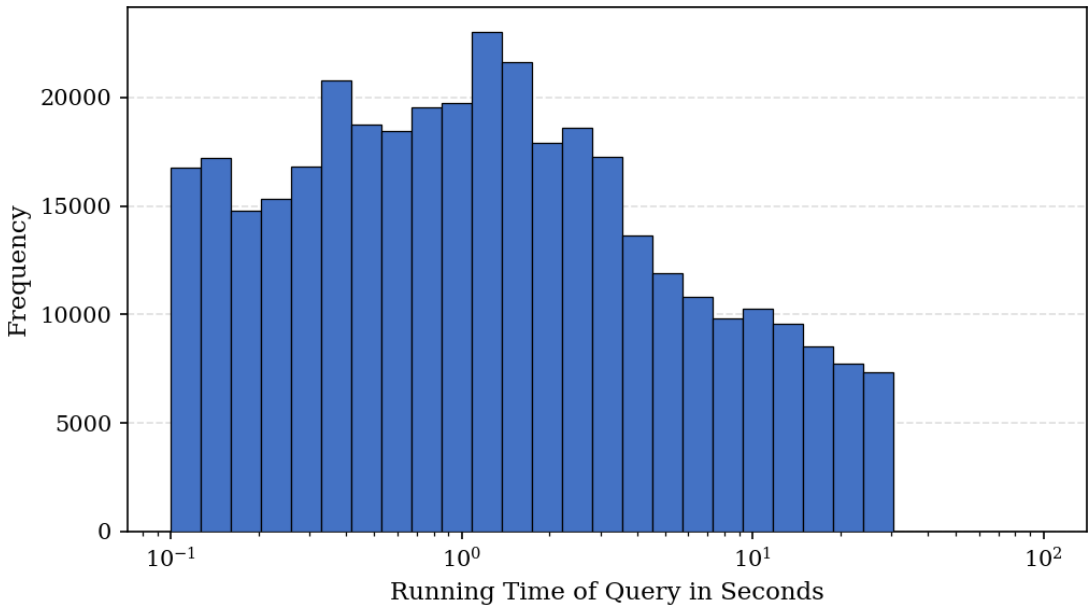

Figure 5.8: Distribution of all query running times of the ZeroShot dataset.

6 Future Work

6.1 Dedicated Benchmarking and Training Data

The most significant limitation of this work is the training data. We rely entirely on the ZeroShot benchmark data, which was designed to enable cost estimation that generalizes to unseen database instances, rather than to cover the full spectrum of PostgreSQL query behavior [**zeroshot**].

The ZeroShot training queries follow a narrow pattern of selective table scans, equi-joins, and a final aggregation to a single tuple [**zeroshot**]. In contrast, T3 generates training data from 16 distinct query structures, explicitly including complex selection predicates and all combinations of filters, joins, aggregations, and sorts [**t3**]. These query types are largely absent from the ZeroShot training data, which limits how well our model can generalize to more diverse workloads.

The JOB comparison was restricted to 77 of the 113 benchmark queries, because the remaining queries exceed the 30 s cutoff of the ZeroShot data [**zeroshot**]. Rieger and Neumann evaluated T3 on all 113 queries [**t3**], making a direct like-for-like comparison on the full JOB benchmark impossible.

Conseqeutively, building a dedicated PostgreSQL benchmark in the style of T3's Umbra benchmark would close the query diversity gap and restore a full JOB comparison.

The runtime distribution of the ZeroShot data is also considerably narrower than T3's Umbra training data [**t3**]. A dedicated benchmark covering a wider runtime range would allow the per-tuple prediction approach to be re-evaluated, since T3's per-tuple normalization is specifically motivated by such a wide range [**t3**] and our ablation study could not test this under comparable conditions (Section 5.3).

Moreover, a data gap is the absence of tablescan input cardinalities in the `parsed_plans`. Our implementation always sets this value to 1, and the model compensates via the many other features. The estimated row count per table is available in PostgreSQL's system catalog (`pg_class.reltuples` [**postgresql_docs_pg_class**]). Future wo should extend the data collection to record this information.

6.2 Feature Vector Extensions

Finer-grained pipeline features. Currently, the pipeline-level block of our feature vector aggregates features across all nodes in a pipeline. For example, `workers_planned` is summed over the pipeline. A natural extension is to expand the pipeline block with per-node entries, at the cost of a larger feature vector. This would let the model distinguish, for example, a pipeline where only one node is parallelized from one where all nodes run in parallel.

Purely query-level feature vector. Per-query prediction outperforms per-pipeline and per-tuple prediction in our setting. Thererefore, a future direction is to design the feature vector directly at the query level, without the intermediate pipeline decomposition. This would further sidestep the PostgreSQL pipeline timing problem identified in the ablation study (Section 5.3), while also simplifying the model pipeline.

Joint pipeline and query context for pipeline-level prediction. For the per-pipeline prediction approach, a possible extension is to provide two feature vectors per pipeline: one capturing only the local pipeline features, and one summarizing the full query context. This gives the model information about how a pipeline relates to the rest of the query, which may improve per-pipeline runtime prediction.

Impact of PostgreSQL cost estimates. We include PostgreSQL planner cost estimates (`est_cost`, `est_startup_cost`) as features. However, the ablation study confirms that the PostgreSQL cost estimator is highly inaccurate for complex queries (median Q-Error above 1000, see Table 5.1). A targeted ablation study that removes or replaces these planner-cost features could determine whether they help or hurt prediction accuracy.

Mitigating the pipeline timing issue. The pipeline timing problem identified in the ablation study arises because PostgreSQL's iterator model causes per-node timings to overlap across pipeline boundaries. Future work could investigate this issue and provide mitigation approaches. Subsequently, this could provide more accurate per-pipeline labels and could make the per-pipeline and per-tuple prediction variants more accurate.

6.3 Prediction Latency

Since both our model and T3 use LightGBM gradient-boosted decision trees, prediction should be fast in practice. Rieger and Neumann note that T3's low prediction latency is

a key practical advantage [**t3**]. Future work should benchmark the prediction latency of our PostgreSQL model to confirm its performance, and to compare it directly against T3's reported latency [**t3**]. A concrete step toward this would be to compile the trained LightGBM model to native machine code using the `lleaves` framework, as T3 does [**t3**]. T3 already demonstrates that this compilation step reduces inference latency from 22 µs to approximately 4 µs, and our model could benefit from the same approach.

7 Conclusion

AI acknowledgen Training Scripts brauchen wir für reproduction ohne MAX

7.1 Section

Citation test [**latex**].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ TUM, TUM

For more details, see the documentation of the `acronym` package[1].

7.1.1 Subsection

See Table 7.1, Figure 7.1, Figure 7.2, Figure 7.3.

Table 7.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

[1] `https://ctan.org/pkg/acronym`

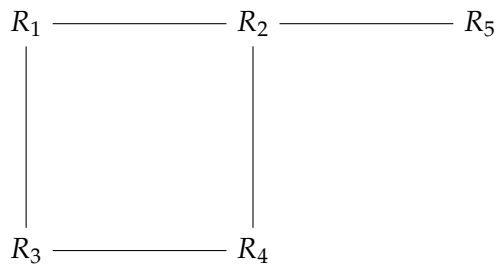

Figure 7.1: An example for a simple drawing.

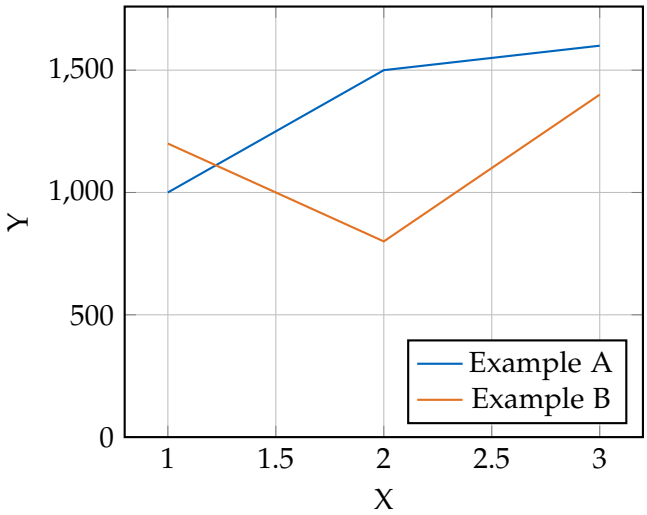

Figure 7.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 7.3: An example for a source code listing.

Abbreviations

TUM Technical University of Munich

List of Figures

1.1 Example drawing 2
1.2 Example plot 2
1.3 Example listing 2

2.1 T3 prediction process 12

4.1 Top-node cutout with Hash Join 23
4.2 Excerpt of a PostgreSQL-native pipeline feature vector 25

5.1 Q-Error across Database Instances 27
5.2 Q-Error Frequency Distribution 28
5.3 57th-Best Q-Error Comparison 29
5.4 Average Q-Error Comparison 29
5.5 57th-Best Q-Error: Our Implementation vs. T3 for PostgreSQL 30
5.6 Cardinality Mode Comparison 32
5.7 Prediction Granularity Comparison 34
5.8 Query Running Time Distribution 35

7.1 Example drawing 40
7.2 Example plot 40
7.3 Example listing 40

List of Tables

1.1 Example table 1

5.1 Median Q-Error on JOB 31

7.1 Example table 39